Differentiable Programming

Concepts of Programming Languages

Dr. David Richter

Winter 2025/26

Software Technology, Computer Science, TU Darmstadt

Today

Learning as Gradient Descent

Automatic Differentiation (AD) Rules

Differentiable Programming - Forward Mode

Differentiable Programming - Reverse Mode

Learning as Gradient Descent

A few Deep Learning Frameworks

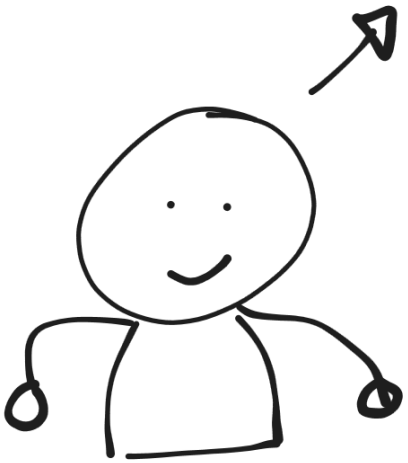
- Torch - since 2002 (Lua)
- Theano - since 2007 (Py)
- Caffee - since 2013 (Py)
- Deeplearning4j - since 2014 (Java)
- Tensorflow - since 2015 (Py)
- PyTorch - since 2016 (Py)
- Jax - since 2018 (Py)

Learning

Learning

Exam for Course "History of Programming"

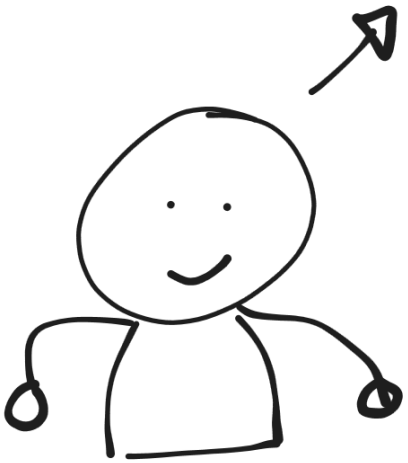
1. When was the lambda calculus invented?
2. When was the turing machine invented?
3. When were Monads invented?
4. When was Java 1.0 published?
5. When did Java get Generics?
6. When was Haskell 1.0 published?
7. When did Haskell get polymorphism?



Learning

Exam for Course "History of Programming"

1. When was the lambda calculus invented?
2. When was the turing machine invented?
3. When were Monads invented?
4. When was Java 1.0 published?
5. When did Java get Generics?
6. When was Haskell 1.0 published?
7. When did Haskell get polymorphism?

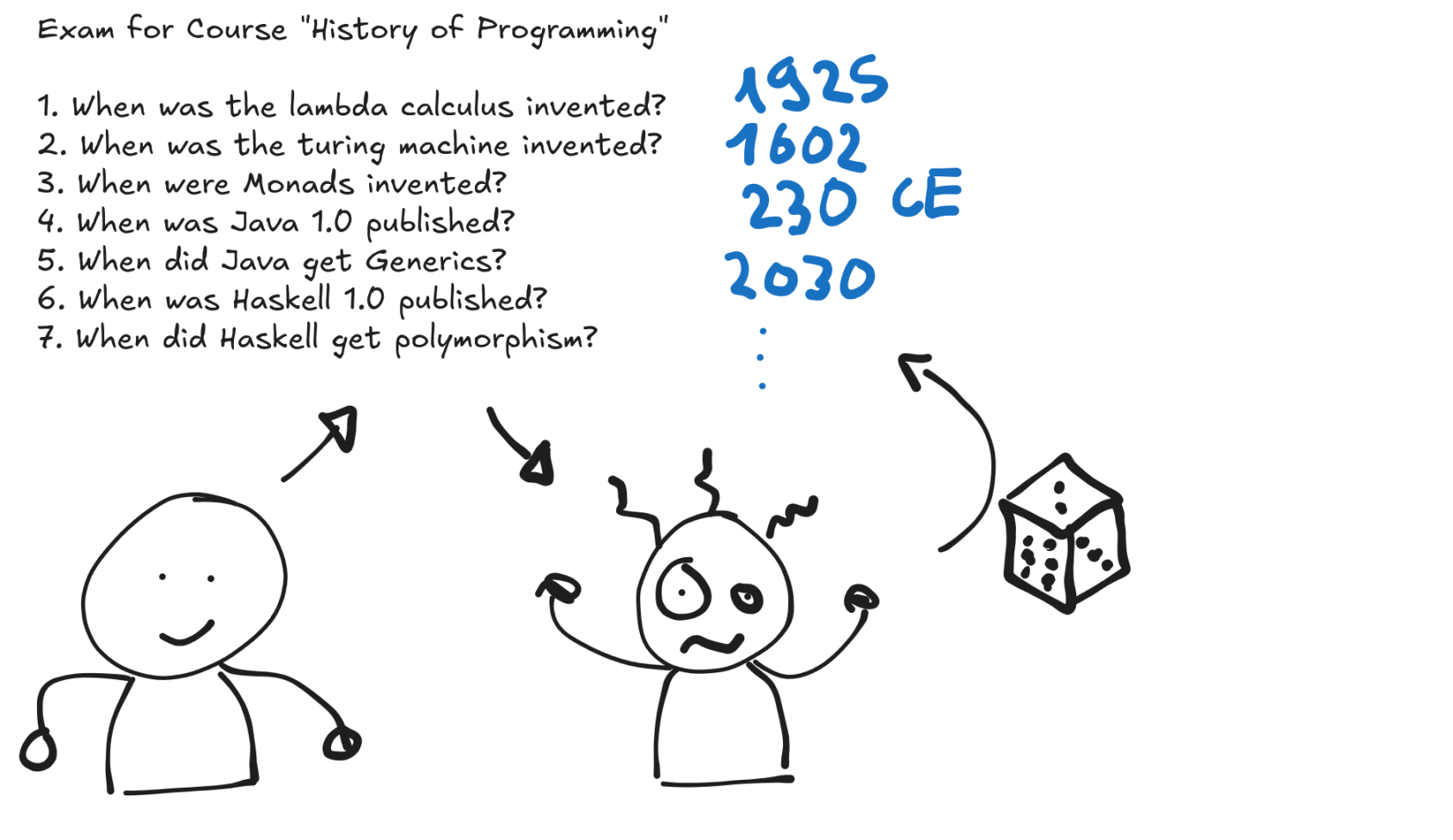


Learning

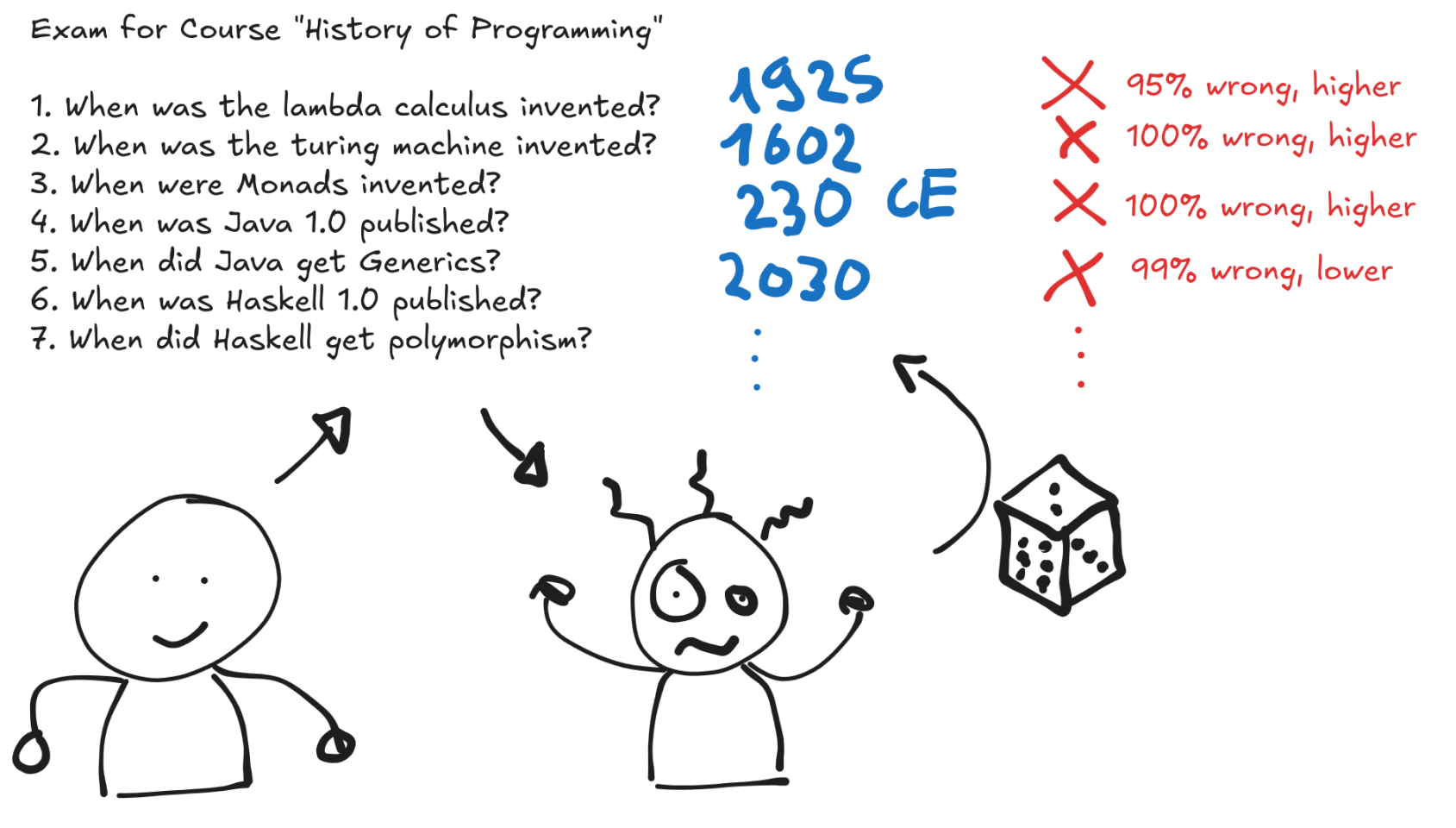
Exam for Course "History of Programming"

1. When was the lambda calculus invented?
2. When was the turing machine invented?
3. When were Monads invented?
4. When was Java 1.0 published?
5. When did Java get Generics?
6. When was Haskell 1.0 published?
7. When did Haskell get polymorphism?

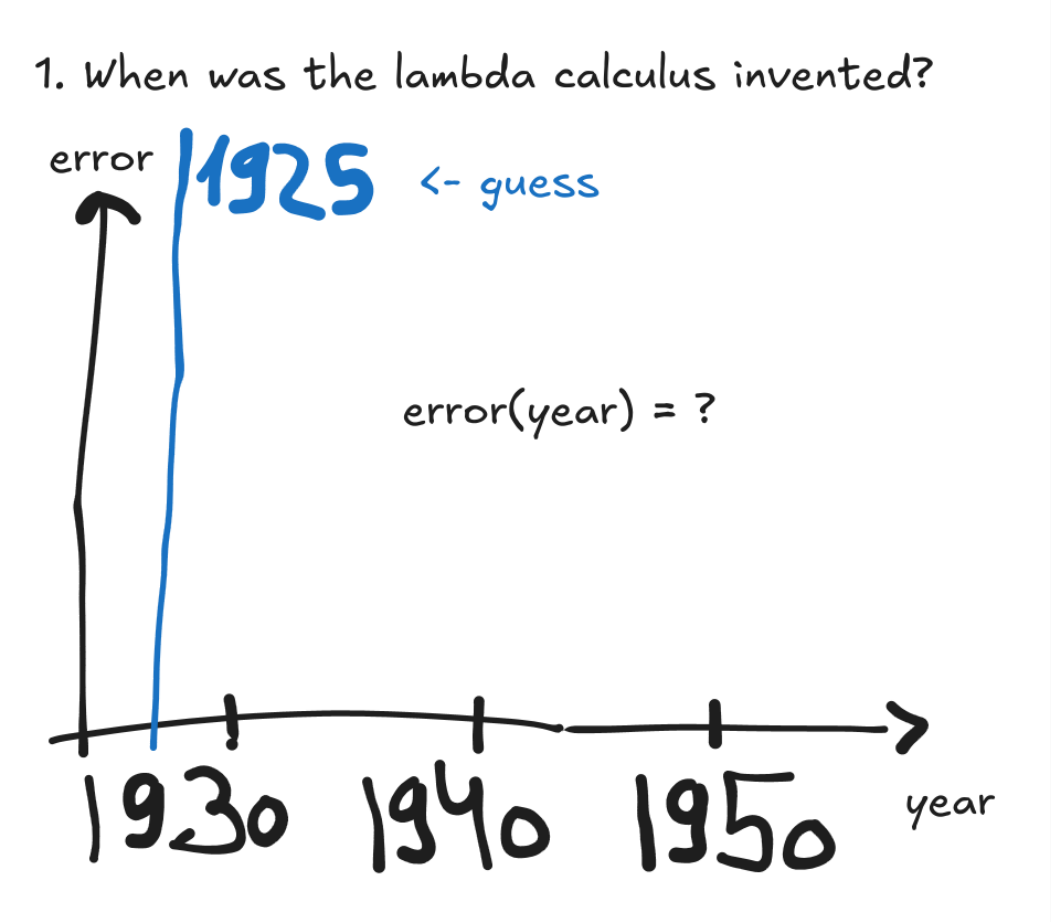
Learning


Exam for Course "History of Programming"
1. When was the lambda calculus invented?
2. When was the turing machine invented?
3. When were Monads invented?
4. When was Java 1.0 published?
5. When did Java get Generics?
6. When was Haskell 1.0 published?
7. When did Haskell get polymorphism?
1925
1602
230 CE
2030

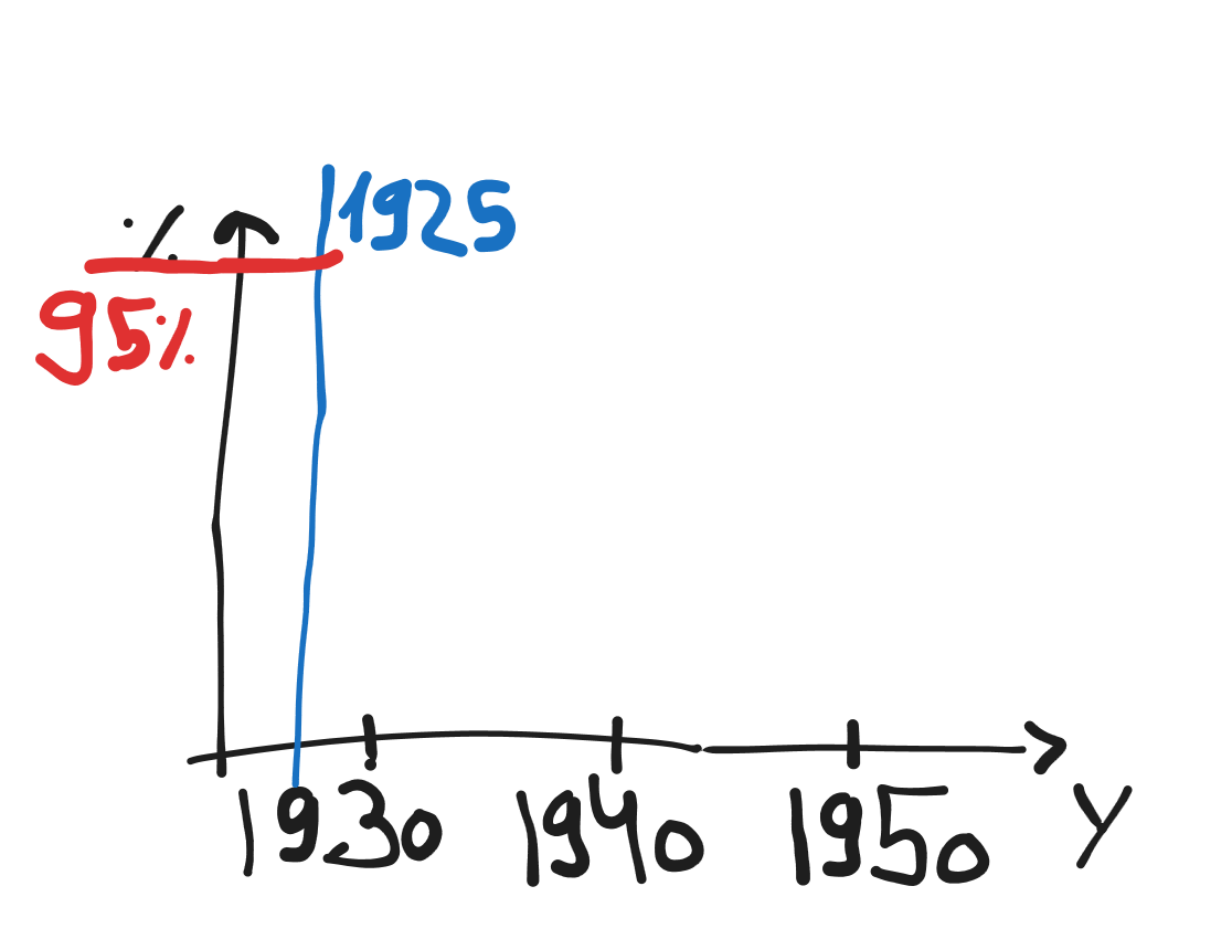
Learning


Exam for Course "History of Programming"
1. When was the lambda calculus invented?
2. When was the turing machine invented?
3. When were Monads invented?
4. When was Java 1.0 published?
5. When did Java get Generics?
6. When was Haskell 1.0 published?
7. When did Haskell get polymorphism?
1925
1602
230 CE
2030
95% wrong, higher
100% wrong, higher
100% wrong, higher
99% wrong, lower

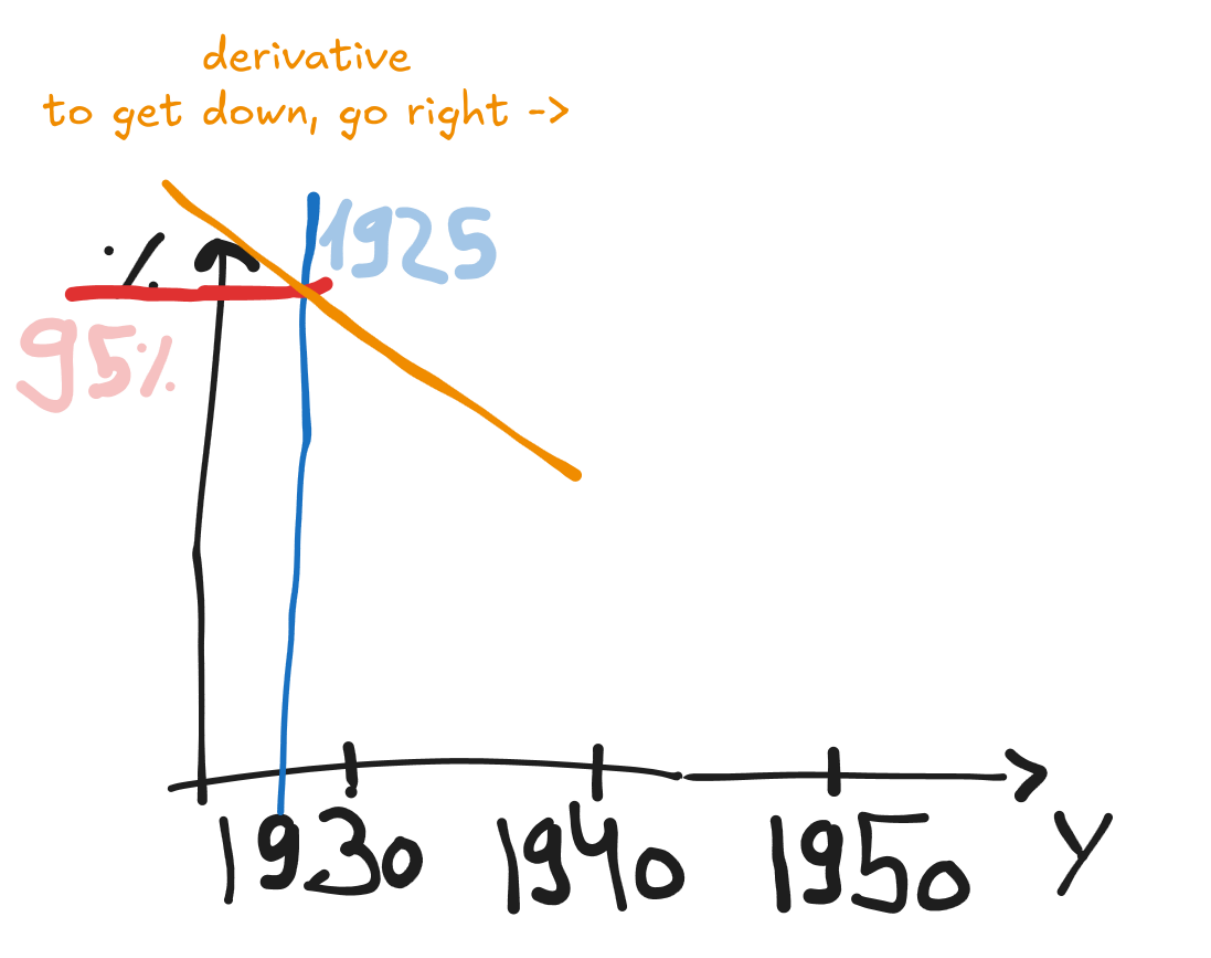
Guessing 1 Question at a Time



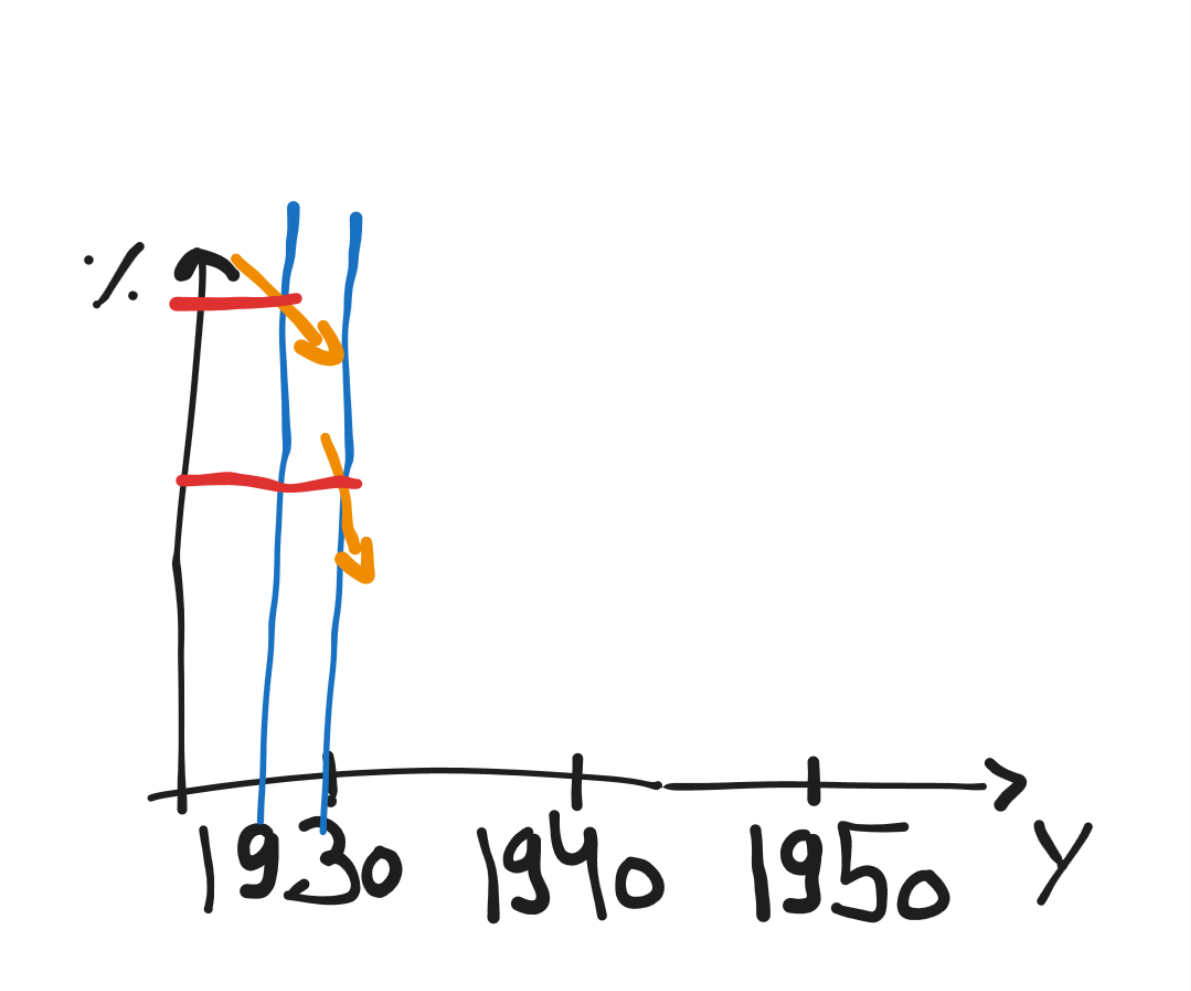
Guessing 1 Question at a Time



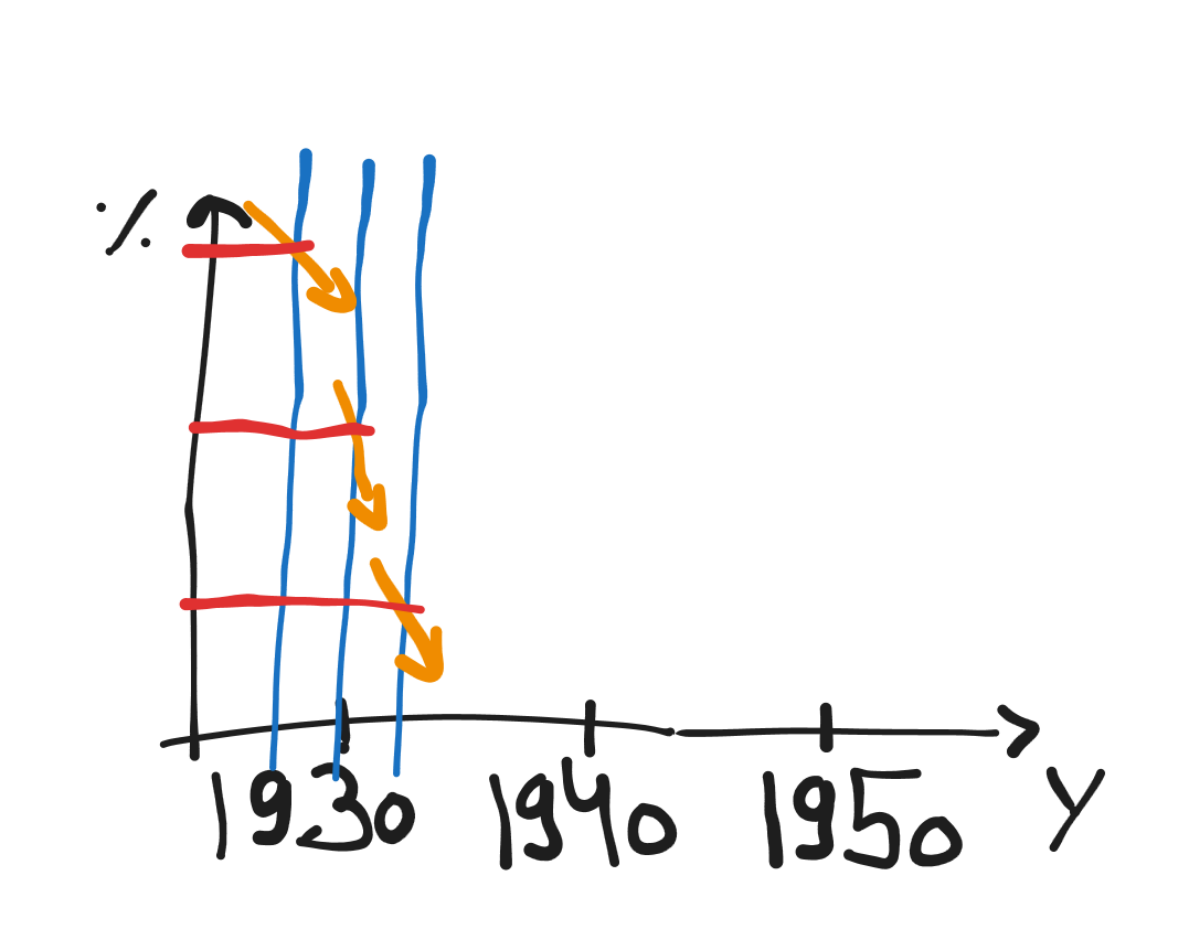
Guessing 1 Question at a Time



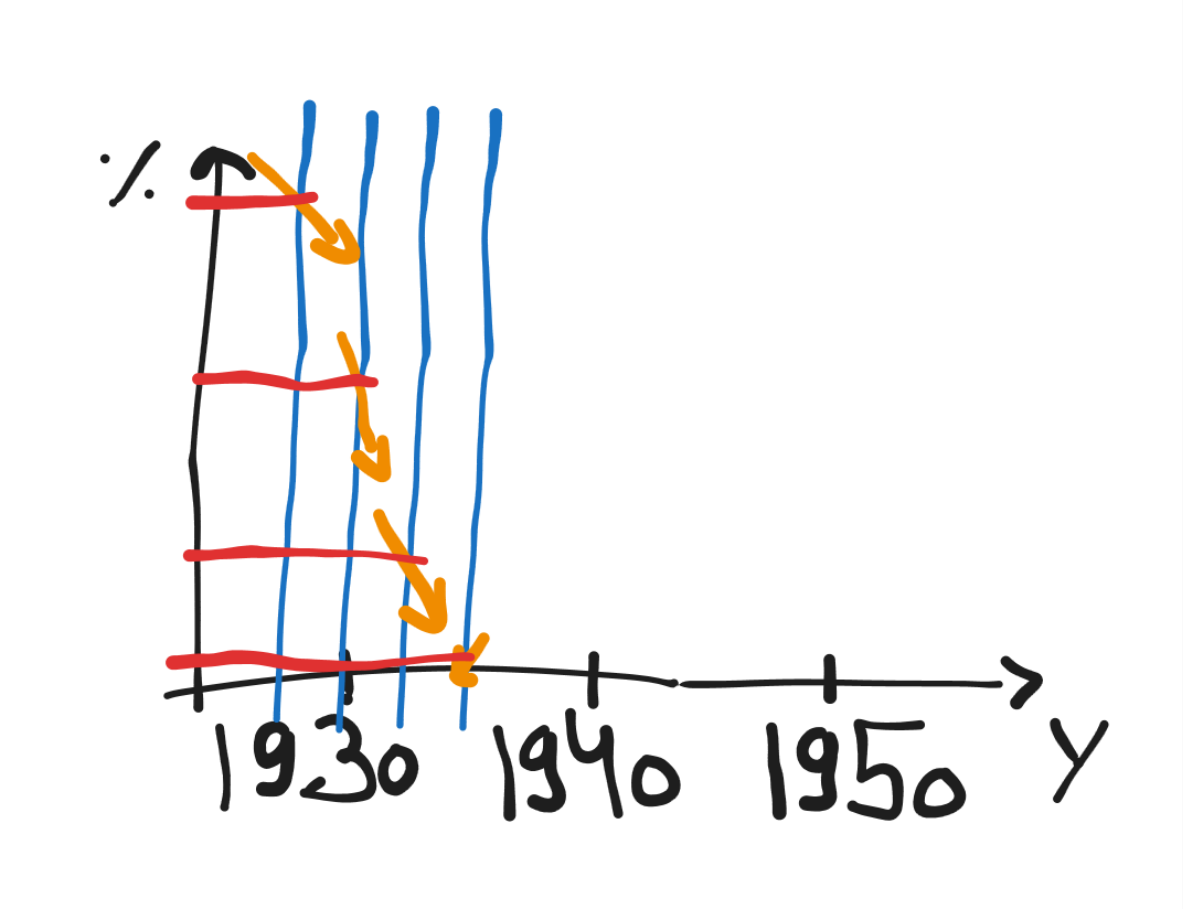
Guessing 1 Question at a Time



Guessing 1 Question at a Time



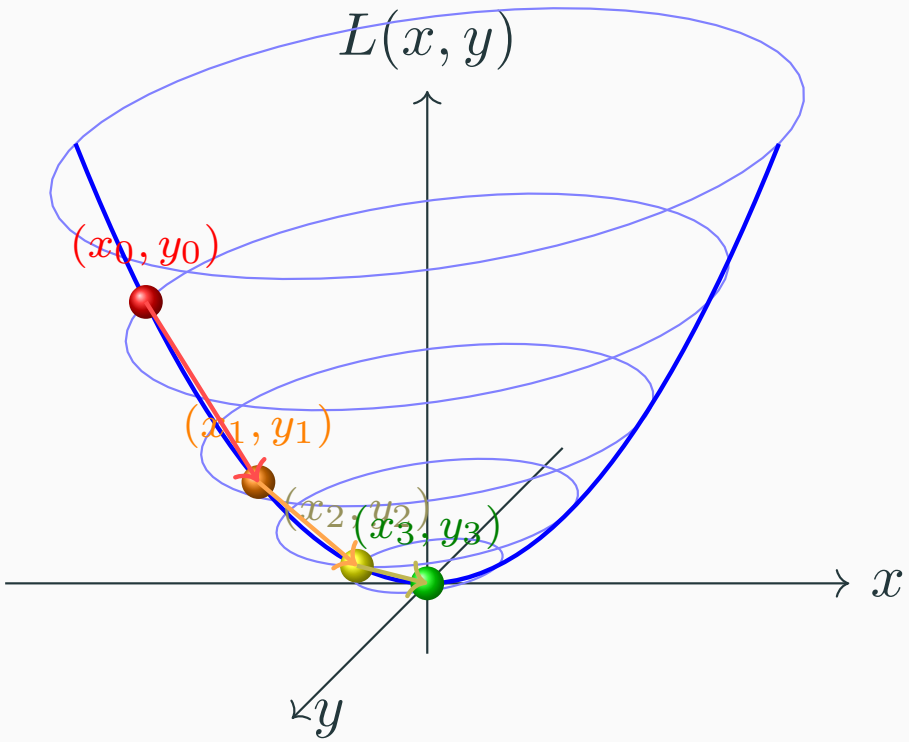
Guessing 1 Question at a Time



Guessing 1 Question at a Time

- Learning is getting better.
- Getting better means minimizing errors.
- The error function tells you how wrong your are.
- Start somewhere, let a ball roll downhill.
- Downhill $\Rightarrow$ Slope $\Rightarrow$ Derivative
- **Gradient Descent**

Guessing 2 Questions at the Same Time



- ball (x_i, y_i) is current guess
- graph maps answer to error $L : R^2 \to R$

Guessing n-Questions at the Same Time

- 1 question error $L : R \to R$
- 2 questions error $L : R^2 \to R$
- n questions error $L : R^n \to R$

Training in Scala

```
def error(w: Double): Double  // user-provided
def deriv(w: Double): Double  // user-provided
val learnrate = 1

var w = 0   // initial guess
for i <- 0 until 1000 do
    val (x, dx) = (error(w), deriv(w))
    w -= learnrate * dx // gradient descent, weight update
    println(s"$epoch: error=${x}, dx=${dx}")
```

Automatic Differentiation (AD) Rules

Programming and Differentiation

A few examples

```
type D = Double
def error(w: D): D = x*x*x + 2*x
def deriv(w: D): D = 3*x*x + 2

def error(w: D): D = sin(x)*exp(x)
def deriv(w: D): D = cos(x)*exp(x)+sin(x)*exp(x)

def error(w: D): D = sqrt(x*x + 1)
def deriv(w: D): D = x / sqrt(x*x+1)

def error(w: D): D = 1 / (1 + exp(-x))
def deriv(w: D): D = val ex = exp(-x); enx / (1 + enx*enx)
```

Kinda annoying

Programming and Differentiation 2

"Deep learning is dead;
long live differentiable programming." — *Yann LeCun*

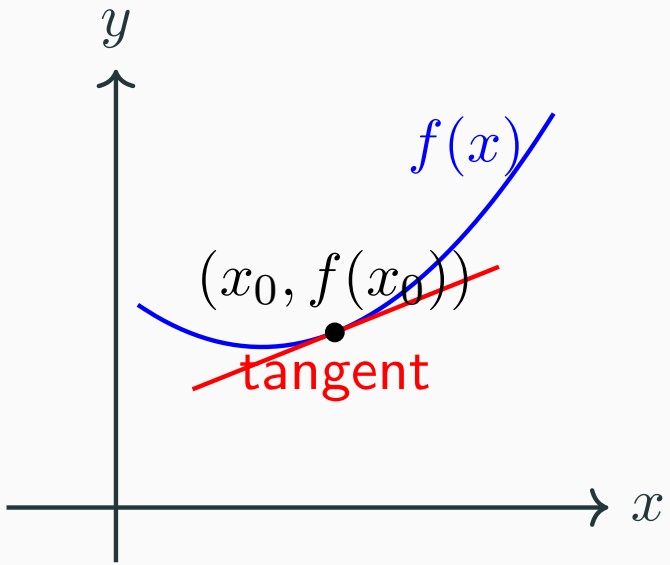
```
def error(w: D): D = 1 / (1 + exp(-x))
def deriv(w: D): D = val ex = exp(-x); enx / (1 + enx*enx)
```

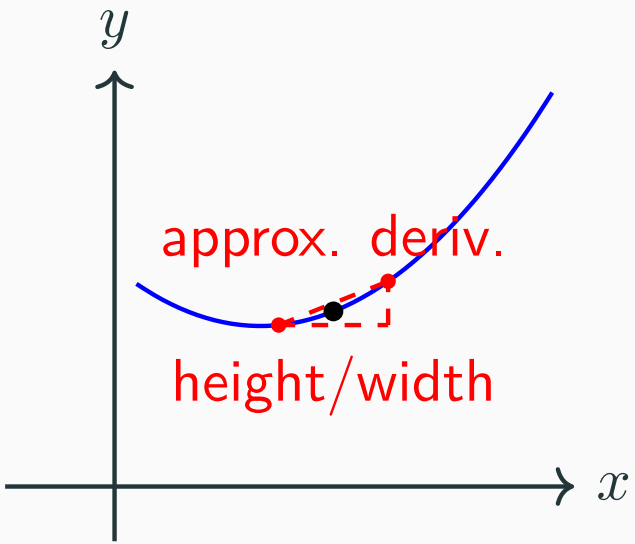
- Writing each derivative manually — tedious and error-prone.
- Loss function could be thousands of lines long.
- Can we derive it automatically from the function?
- That's the essence of **automatic differentiation** (AD).

Approximative AD

- The derivative at a point is the slope of the tangent line.
- It shows how fast a function changes at that point.
- Let's approximate, via height/width of a small triangle

$$f'(x) = \lim_{\epsilon \to 0} \frac{f(x+\epsilon) - f(x-\epsilon)}{2\epsilon}$$





Approximative AD Scala

```
val eps: Double = 1e-5
def deriv(f: Double => Double, x: Double): Double =
  f(x + eps) - f(x - eps)) / (2 * eps)

def f(x: Double): Double = 0.3*x*x - 0.8*x + 2
for x0 <- 0 until 20 do
  println(s"f($x0) = ${f(x0)}; f'($x0) ~ ${deriv(f, x0)}")
```

- $f'(x) = \lim_{\epsilon \to 0} \frac{f(x+\epsilon) - f(x-\epsilon)}{2\epsilon}$

Approximative AD Scala

```
val eps: Double = 1e-5
def deriv(f: Double => Double, x: Double): Double =
  f(x + eps) - f(x - eps)) / (2 * eps)

def f(x: Double): Double = 0.3*x*x - 0.8*x + 2
for x0 <- 0 until 20 do
  println(s"f($x0) = ${f(x0)}; f'($x0) ~ ${deriv(f, x0)}")
```

- $f'(x) = \lim_{\epsilon \to 0} \frac{f(x+\epsilon) - f(x-\epsilon)}{2\epsilon}$
- Approximative AD is easy to implement, but quite unstable!

Approximative AD Scala

```
val eps: Double = 1e-5
def deriv(f: Double => Double, x: Double): Double =
  f(x + eps) - f(x - eps)) / (2 * eps)

def f(x: Double): Double = 0.3*x*x - 0.8*x + 2
for x0 <- 0 until 20 do
  println(s"f($x0) = ${f(x0)}; f'($x0) ~ ${deriv(f, x0)}")
```

- $f'(x) = \lim_{\epsilon \to 0} \frac{f(x+\epsilon) - f(x-\epsilon)}{2\epsilon}$
- Approximative AD is easy to implement, but quite unstable!
- the longer the function is, the more the approximation deviates from the actual result

Symbolic AD

Derivatives can be automated!

$$(c)' = 0 \qquad \textit{(constant; no x in c)}$$

$$(a+b)' =$$

$$(a \cdot b)' =$$

$$\left(\frac{a}{b}\right)' =$$

$$(a^n)' =$$

$$(\sin a)' =$$

$$(\cos a)' =$$

Symbolic AD

Derivatives can be automated!

$$(c)' = 0 \qquad \textit{(constant; no x in c)}$$

$$(a+b)' = a' + b'$$

$$(a \cdot b)' =$$

$$\left(\frac{a}{b}\right)' =$$

$$(a^n)' =$$

$$(\sin a)' =$$

$$(\cos a)' =$$

Symbolic AD

Derivatives can be automated!

$$
\begin{aligned}
(c)' &= 0 && (\textit{constant; no } x \textit{ in } c) \\
(a+b)' &= a' + b' \\
(a \cdot b)' &= a' \cdot b + a \cdot b' \\
\left(\frac{a}{b}\right)' &= \\
(a^n)' &= \\
(\sin a)' &= \\
(\cos a)' &=
\end{aligned}
$$

Symbolic AD

Derivatives can be automated!

$$
\begin{aligned}
(c)' &= 0 && (\textit{constant; no } x \textit{ in } c) \\
(a+b)' &= a' + b' \\
(a \cdot b)' &= a' \cdot b + a \cdot b' \\
\left(\frac{a}{b}\right)' &= \frac{a' \cdot b - b \cdot a'}{b^2} \\
(a^n)' &= \\
(\sin a)' &= \\
(\cos a)' &=
\end{aligned}
$$

Symbolic AD

Derivatives can be automated!

$$
\begin{aligned}
(c)' &= 0 && (\textit{constant; no x in c}) \\
(a+b)' &= a' + b' \\
(a \cdot b)' &= a' \cdot b + a \cdot b' \\
\left(\frac{a}{b}\right)' &= \frac{a' \cdot b - b \cdot a'}{b^2} \\
(a^n)' &= n \cdot a^{n-1} \cdot a' \\
(\sin a)' &= \\
(\cos a)' &=
\end{aligned}
$$

Symbolic AD

Derivatives can be automated!

$$
\begin{aligned}
(c)' &= 0 && (\textit{constant; no x in c}) \\
(a+b)' &= a' + b' \\
(a \cdot b)' &= a' \cdot b + a \cdot b' \\
\left(\frac{a}{b}\right)' &= \frac{a' \cdot b - b \cdot a'}{b^2} \\
(a^n)' &= n \cdot a^{n-1} \cdot a' \\
(\sin a)' &= +\cos a \cdot a' \\
(\cos a)' &=
\end{aligned}
$$

Symbolic AD

Derivatives can be automated!

$$
\begin{aligned}
(c)' &= 0 && (\textit{constant; no x in c}) \\
(a+b)' &= a' + b' \\
(a \cdot b)' &= a' \cdot b + a \cdot b' \\
\left(\frac{a}{b}\right)' &= \frac{a' \cdot b - b \cdot a'}{b^2} \\
(a^n)' &= n \cdot a^{n-1} \cdot a' \\
(\sin a)' &= +\cos a \cdot a' \\
(\cos a)' &= -\sin a \cdot a'
\end{aligned}
$$

Differentiable Programming - Forward Mode

Dual Numbers

```
// instead of two functions
def error(x: Double): Double  = 2*x + x*x
def deriv(x: Double): Double = 2 + 2*x

// a single function, doing both
case class Dual(x: Double, dx: Double)
def errorDual(x: Dual): Dual =
  add(mul(const(2), x), mul(x, x))

// error(x) == errorDual(x).x
// deriv(x) == errorDual(x).dx
```

- A `Dual` holds both the value x and its derivative dx.
- Operations on `Duals` follow the rules of calculus automatically.

What is a Number?

```
trait Number[X]:
  def const(c: Double): X
  def add(a: X, b: A): X
  def mul(a: X, b: A): X
  def div(a: X, b: A): X
  def pow(a: X, n: Int): X
  def sin(a: X): X
  def cos(a: X): X
```

Double is a Number

```
given Number[Double] with
  def const(c: Double): Double = c
  def add(a: Double, b: Double): Double = x + y
  def mul(a: Double, b: Double): Double = x * y
  def div(a: Double, b: Double): Double = x / y
  def pow(a: Double, n: Int): Double = Math.pow(x, n)
  def sin(a: Double): Double = Math.sin(x)
  def cos(a: Double): Double = Math.cos(x)
```

Dual is also a Number

```
given Number[Dual] with
  def const(c: Double): Dual = ...
  def add(a: Dual, b: Dual): Dual = ...
  def mul(a: Dual, b: Dual): Dual = ...
  def div(a: Dual, b: Dual): Dual = ...
  def pow(a: Dual, n: Int): Dual = ...
  def sin(a: Dual): Dual = ...
  def cos(a: Dual): Dual = ...
```

... on the next slides

Dual Rule — Constant

$$(c)' = 0$$

```
def constant(c: Double): Dual =
  Dual(c,
       0 )
```

Dual Rule — Addition

$$(a + b)' = a' + b'$$

```
def add(a: Dual, b: Dual): Dual =
  Dual(a.x + b.x,
       a.dx + b.dx )
```

Dual Rule — Multiplication

$$(a \cdot b)' = a' \cdot b + a \cdot b'$$

```
def mul(a: Dual, b: Dual): Dual =
  Dual(a.x * b.x,
       a.dx * b.x + a.x * b.dx)
```

Dual Rule — Division

$$\left(\frac{a}{b}\right)' = \frac{a' \cdot b - a \cdot b'}{b^2}$$

```
def div(a: Dual, b: Dual): Dual =
  Dual(a.x / b.x,
       ???) // exercise
```

Interpreter

In the class, we usually implement stuff as an interpreter. Above we sketched how to implement it as a library.

Differentiable Programming as an interpreter is left as an exercise! (Instead of writing a library, just put the rules into the interpretation function.)

Differentiable Programming - Reverse Mode

Forward vs Reverse Mode AD — Intuition

- Remember, for learning n-questions, the error function has type $R^n \to R$
- To get the direction to move, we need the derivatives in all directions of the function

$$f : \mathbb{R}^n \to \mathbb{R}$$

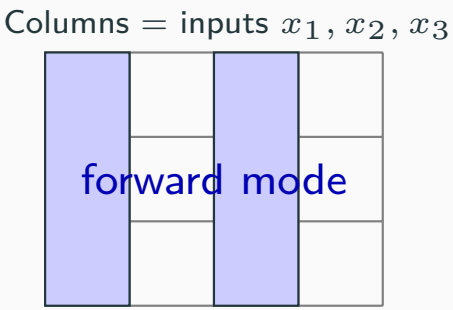
- The **Jacobian** J of f at position $x_1, ..., x_n$ has n columns:

$$J_f(x) = \begin{bmatrix} \frac{\partial f(x_1,...,x_n)}{\partial x_1} & \dots & \frac{\partial f(x_1,...,x_n)}{\partial x_n} \end{bmatrix}$$

- One call of the dual number function calculates one entry.
- Reverse Mode calculates all entries in a single call.

Forward Mode — Column by Column

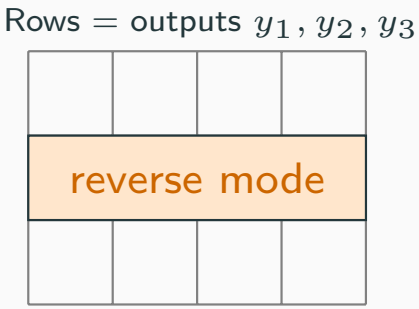
- In forward mode, we propagate derivatives of intermediate values with respect to one input x_i.
- Each evaluation computes one **column** of the Jacobian.



computing $\frac{dy}{dx_1}$ for all outputs y_k.

Reverse Mode — Row by Row

- In reverse mode, we backpropagate the influence of intermediate values with respect to output y_j.
- Each backward pass computes one **row** of the Jacobian.



computing $\frac{\partial y_1}{\partial x}$, the gradient of one output.

Forward vs Reverse Mode

- In our case the error function has only a single output, and thus there is only a single row.
- Thus, reverse mode is more efficient, calculating the whole Jacobian and the direction to go in a single step.

Derivative Rules are Linear

Observe: Derivatives are linear; linear functions can be written as:

$$y' = c \cdot x'$$

A linear function with multiple inputs has form:

$$y' = c_0 \cdot x'_0 + c_1 \cdot x'_1 + c_2 \cdot x'_2 + ...$$

Derivative Rules are Linear

Each rule can be written in form $y' = c \cdot x'$

$$
\begin{aligned}
(c)' &= 0 &&= 0 \cdot x \\
(a+b)' &= a' + b' &&= 1 \cdot a' + 1 \cdot b' \\
(a \cdot b)' &= a' \cdot b + a \cdot b' &&= b \cdot a' + a \cdot b' \\
\left(\frac{a}{b}\right)' &= \frac{a'b - ab'}{b^2} &&= \frac{1}{b} \cdot a' + \frac{-a}{b^2} \cdot b' \\
(a^n)' &= na^{n-1}a' &&= na^{n-1} \cdot a'
\end{aligned}
$$

...

Example

- How get we systematically figure out what the reverse mode rules are?
- 1) an example program
- 2) A tape storing its derivative in forward mode with usual derivative rules.
- 3) To evaluate the reverse mode, we transpose each line separately, e.g., `dy += c * dx` becomes `dx += c * dy`
- Finally, watch out, that all writes to a variable appear before all reads, e.g., we need to sort the lines (topogically).

```
// original

def foo(x,y) =
  val z = mul(x, y)
  val w = add(x, z)
  w
```

```
// forward derivative tape
// for readability we use strings as names,
// in the exercise we just enumerate them
List(
  TapeEntry("dz", Add(Scale(y, "dx"), Scale(x, "dy"))),
  TapeEntry("dw", Add("dx", "dz")),
)
```

```
// to evaluate tape in reverse mode: forward mode rules, but each line transposed
val state = new mutable.HashMap[String, Double]()
... // init with zero
state(dx) += 1 * state(dw); state(dz) += 1 * state(dw) // from dw =   dx +   dz
state(dx) += y * state(dz); state(dy) += x * state(dz) // from dw = y*dx + x*dy
return state(dx), state(dy)
```

Reverse Mode

```
// a tape is a list of instructions
type Tape = List[TapeEntry]
// each instruction is of form "dy += c * dx"
case class TapeEntry(y: TapeId, d: Double, x: TapeId)

// the interpreter is changed to return a value and a state
def interp(expr: FCRV, env: Env, state: State): WithState =
  expr match
  // in each case in the interpreter,
  // the tape is used to track the performed operation
  // ...

// with state includes
case class WithState(v: Value, s: State)
case class State(nextId: TapeId, tape: Tape):
```

The end

- Learning is getting better
- Minimize error function
- Forward mode dual numbers calculate both value and derivative
- Reverse mode uses a tape to track operations, and calculate whole derivative in a single swoop